



To PIM or Not for Emerging General Purpose Processing in DDR Memory Systems

Alexandar Devic
devic@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Siddhartha Balakrishna Rai
sub1089@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Anand Sivasubramaniam
axs53@psu.edu
The Pennsylvania State University
State College, Pennsylvania, USA

Ameen Akel
aakel@micron.com
Micron Technology, Inc
Folsom, California, USA

Sean Eilert
sseilert@micron.com
Micron Technology, Inc
Folsom, California, USA

Justin Eno
jeno@micron.com
Micron Technology, Inc
Folsom, California, USA

ABSTRACT

As Processing-In-Memory (PIM) hardware matures and starts making its way into normal compute platforms, software has an important role to play in determining what to perform where, and when, on such heterogeneous systems. Taking an emerging class of PIM hardware which provisions a general purpose (RISC-V) processor at each memory bank, this paper takes on this challenging problem by developing a software compilation framework. This framework analyzes several application characteristics – parallelizability, vectorizability, data set sizes, and offload costs – to determine what, whether, when and how to offload computations to the PIM engines. In the process, it also proposes a vector engine extension to the bank-level RISC-V cores. Using several off-the-shelf C/C++ applications, we demonstrate that PIM is not always a panacea, and a framework such as ours is essential in carefully selecting what needs to be performed where, when and how. The choice of hardware platforms – number of memory banks, relative speeds and capabilities of host CPU and PIM cores, can further impact the “to PIM or not” question.

CCS CONCEPTS

• **Computer systems organization** → **Heterogeneous (hybrid) systems**; • **Hardware** → **Emerging architectures**; **Emerging languages and compilers**.

KEYWORDS

Processing-in-Memory, DRAM, General Purpose Processing, Compilers, Vector Processing, Parallel Processing

ACM Reference Format:

Alexandar Devic, Siddhartha Balakrishna Rai, Anand Sivasubramaniam, Ameen Akel, Sean Eilert, and Justin Eno. 2022. To PIM or Not for Emerging General Purpose Processing in DDR Memory Systems. In *The 49th Annual International Symposium on Computer Architecture (ISCA '22)*, June 18–22,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

ISCA '22, June 18–22, 2022, New York City, NY

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-8610-4/22/06...\$15.00

<https://doi.org/10.1145/3470496.3527431>

2022, New York, NY, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3470496.3527431>

1 INTRODUCTION

The memory wall has necessitated the exploration of novel architectures to bridge the physical divide between memory and compute engines. Bringing compute elements close to memory storage structures can not only reduce the latency for data accesses, but also help boost parallelism – since each compute element can access the storage structure close to it without contending with others over a single serial interface to the memory system. Recognizing these needs, there has been a plethora of Processing-in-Memory (PIM) hardware proposed, ranging from (i) those that perform specialized/hardwired operations [58] to those that are more generally programmable [7, 20, 51–53], (ii) those that offer immense (several thousand fold) parallelism with very simple logic structures operating at individual bit/byte levels [23, 58] to those that work with entire banks of memory [20, 53], and (iii) those that work with specialized memory technologies such as HBM [3, 50], ReRAM [16, 30, 43], etc., to those that work with commoditized DDR DRAM technology [20, 51, 53, 67]. Despite these hardware proposals, software is still not fully mature to figure out what, where, and how to take off-the-shelf applications and map them on to PIM engines for maximum effectiveness. Taking an important class of emerging PIM technologies which incorporates general purpose programmable processors within each DDR memory bank [7, 20], this paper tries to fill this gap by developing a software framework to answer the what (identifying regions of code from the full application), where (on the main cores or on PIM cores, or on vector engines), and how (figuring out how to transfer the input data and output results) questions for off-the-shelf applications. We demonstrate that PIM is not always a panacea, and the what, where, and how capabilities of our software framework are critical to realize its full benefits.

Over the years, there have been numerous PIM proposals that differ in their specialization, programmability, granularity, and memory technology used. Much of this has been at the research level, and full-fledged PIM based memory chips have not been fully commercialized. This is largely due to the commodity memory system market, where cost and performance are key drivers for widespread adoption. An additional impediment is the customized hardware for specialized operation in many of these prior proposals [23, 42, 57, 60] which may not justify wide applicability across

multiple workloads. It is only recently [20] that “general purpose programmable processors” have been incorporated into memory banks of the widely prevalent DDR-based memory systems, with commercial offerings of this product starting to make its way into the market. We focus on such PIM systems in this paper, examining the software framework required to map off-the-shelf applications on to these platforms. Extending these ideas for other PIM designs, and/or accelerators in general, is beyond the scope of this paper.

There are opportunities and challenges in leveraging these emerging PIM designs. On the one hand, with a general purpose processor (such as a RISC-V[5] core in the UPMEM product [7, 20]), one can leverage hundreds of these cores placed close to each DDR bank that can work in parallel, availing of lower latency to access data in its proximate bank. However, these designs also suffer from the following drawbacks: (i) the PIM cores are typically much weaker in processing power (clock speeds, floating point capabilities, etc.) compared to the more powerful (say Intel Xeon class) processors at the host end, and (ii) the proximate location within the bank restricts what each of these PIM cores can access (only the data within its bank) because of DDR nuances, and (iii) the restriction on what each PIM core can access, requiring that the host core (re-)layout the input (and subsequently the output) to (from) the original DDR-optimized layout for the host to the local bank of each PIM core, thereby incurring additional costs. Any application execution should carefully weigh these benefits and drawbacks to determine whether, when, and how to offload specific code regions on to these PIM engines, rather than run them on the host cores themselves. We do not want to burden application programmers either, with these intricate details to complicate their tasks further, and would like an automated framework to make these decisions. To date, there is no such sophisticated (software) framework to determine whether “to PIM or not?”, for these emerging designs.

Additionally, today’s server processors provide hardware mechanisms at or near the host cores to accelerate many data intensive operations. One such common mechanism is vectorization, wherein vector engines can fetch large arrays to perform repetitive operations on individual elements, using a single vector instruction rather than having separate fetch-decode-execute instruction repetitions for each vector element. Such vector engines, such as SSE and AVX [1, 4], are common in today’s high end processors. This introduces another dimension to the “PIM or not” question – it may be sometimes more efficient to execute certain regions of code on these engines, as opposed to the main cores, or the PIM cores at the memory bank.

While today’s commercial PIM cores are simple scalar engines, one could envision vector engines being incorporated into those cores as well. In fact, such vector engines would additionally benefit from the relatively wide data paths to the memory bank – one could even potentially access the entire row buffer simultaneously to perform vector operations in parallel, as opposed to normal DDR channels which are only 64 bits (8 bytes) wide. Hence, in this study, we also consider the hardware design for incorporating vector engines at these PIM cores, and study their potential benefit for an offload capability within our software framework.

To our knowledge this paper presents the first such software framework to answer the what, where, and how questions for accelerating off-the-shelf application code onto programmable processors at DDR memory banks. We discuss and propose a PIM engine and implement a software framework for said architecture using the LLVM compilation framework, and evaluate its effectiveness using select off-the-shelf SPEC 2017[14] applications, and Map-Reduce datacenter benchmarks from the Phoenix [2] suite. Specifically, this paper makes the following contributions:

- Identify the dimensions (parallelization, vectorization, and offload costs) needed to determine *what* regions of code to run *where* – host core(s), host vector engine(s), PIM bank cores, or PIM bank vector engines.
- Architect a framework to track and quantify these dimensions at compile time and provide the *how* (if at all) to offload and run the different regions of the application.
- Incorporating a unified *Bank Level In-Memory* (general purpose) Processor architecture (such as UPMEM[20]), termed *BLIMP*, while extending these scalar technologies with vector engines that leverage wider channel access to the proximate bank row buffers, which we define as *BLIMP-Vector* or *BLIMP-V*.
- Building an evaluation framework that uses a combination of (i) an actual machine with Xeon class processors and AVX vector engines to model the host side execution, and (ii) a cycle accurate simulation leveraging a combination of DRAMSim2 [56] and riscvOVPsim [6] to model memory accesses and RISC-V cores at the bank. This evaluation platform and compilation framework will be open-sourced for the broader research community.
- Evaluating the need and effectiveness of such a software platform by using 9 C/C++ applications from SPEC 2017[14], and 7 Map-Reduce datacenter-driven applications from Phoenix [2].

2 BACKGROUND

On the computation side, this work mainly employs two kinds of computation engines: (i) General purpose processors at the host (Xeon class) and/or at DRAM banks (RISC-V cores); and (ii) Vector engines specifically intended to operate on large arrays by coalescing repetitive instructions into single vector instructions. Currently, most host processors include vector engines (such as AVX [4]) to support repetitive operations on array elements. While we are already witnessing the emergence of RISC-V cores proximate to each DRAM bank [3, 7, 67], there is no such vectorization capability on recently announced PIM products [3, 7]. For fairness, we also consider such a capability at the DRAM bank – with the details of its hardware implementation discussed in the next section. In this work, we do not consider more specialized accelerators on the host, nor customized hardware that can work on storage arrays within the bank such as other numerous PIM proposals [57, 58]. We are limiting ourselves to programmable general-purpose processors at both host and DRAM ends, wherein the software has to make more intelligent choices on what and where to execute a given application.

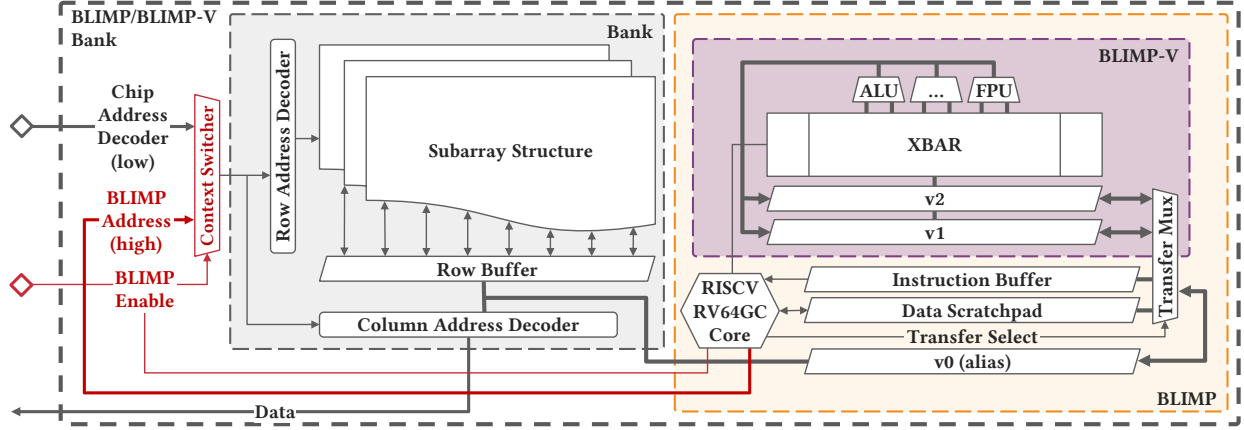


Figure 1: Architecture of a BLIMP and BLIMP-V Bank.

2.1 DDR DRAM Organization

Since this work depends on an understanding of the memory organization of DDR systems, we briefly review the concepts here. Current memory systems utilize different technologies such as DDR/DIMM, HMC, HBM, etc. Irrespective of the choice of technology, they are organized as a hierarchy of storage structures. Of these, DDR is the most prevalent, especially due to its cost-effectiveness (per gigabyte) in off-the-shelf servers for datacenters, hence is used as our baseline to which we add PIM capabilities.

Compute systems access their memory systems through multiple channels. Each channel is provided with a separate address, data, and command bus which enables concurrent and parallel memory access. Multiple Dual In Memory Modules (or DIMM)s are then plugged into the slots provided on the motherboard for each channel. Each DIMM is composed of multiple ranks with each rank having a set of DRAM devices, which we will henceforth refer to as chips. Chips are composed of banks which may vary in number from 8 to 16. Though the bank is itself further composed of sub-arrays, for the purposes of this study we do not delve deeper and we confine ourselves to viewing the bank as a 2D array of DRAM cells. Each bank is provided with a row buffer, and in our case we assume a standard row buffer width of 1KB, or 8192 bits. When an address is generated, the entire row associated with that address is loaded into the row buffer, and then the specific bytes associated with that address are supplied from the row buffer to higher levels.

2.2 Traditional Address Mappings and Data Layout

As a DIMM manufacturer, striping the data across channels, ranks, and banks in a variety of permutations offers boosts in bandwidth by exploiting rank level or bank level parallelism. However, regardless of the mapping chosen, within the rank of a channel, the data is striped across chips. In our study, we are assuming a rank to possess 8 “x8” chips. “x8” means that each chip will provide 8 bits of data at a time. For example, consider a 64 bit value, the first byte is stored in Chip 0, the 2nd byte in Chip 1, and so on. However,

this mapping, though valid, is purely for the purpose of demonstration. In fact, various mapping strategies have been employed which differ from this simplistic scheme and are aimed specifically at improving spatial locality [39] and reducing row-buffer conflicts [66]. A detailed explanation of these various strategies is beyond the scope of this paper.

Regardless, each “x8” chip provides 8 bits of data, and the rank as a whole provides 64 bits of data to fill the 64 bit data bus that interfaces the rank to the memory controller. Data, however, is fetched at a cache-line granularity, i.e 64 bytes. Therefore data is fetched with a burst length of 8 ($8 \times 8B = 64B$).

For the remainder of this paper, we assume a DRAM organization and architecture which supports 8 banks per chip, each bank containing a 1KB row buffer and 32 MB of data capacity. Consequently, on such a system, if a byte at address x falls on a bank of a chip, the byte at address $x + 1$ falls on the corresponding bank of the next chip and so on. It is only the byte at address $x + 8$ that falls again on the same bank of the same chip. The program running on the host does not explicitly see this mapping/stripping across chips/banks.

3 HARDWARE PLATFORMS

The current baseline platforms we consider with an off-the-shelf DDR DRAM system include (i) a single core processor (termed CPU), (ii) a multi-core (termed MTCPU) such as an Intel Xeon Gold, and (iii) each of these with vector engines, such as AVX [4]. Below, we describe the PIM hardware at the DRAM end, either with just a normal general purpose RISC-V core (as in UPMEM [20]), termed BLIMP, or a RISC-V core coupled with a vector engine, termed BLIMP-V.

3.1 BLIMP: Bank-Level In-Memory Processing

Representative of recent hardware that incorporates a general purpose programmable processor at each DRAM bank, our BLIMP platform has a modified and stripped 200 MHz RISC-V RV64GC processor at the bank. This single-stage in-order processor is stripped of all traditional cache, tables, and buffers and instead fitted with an instruction buffer and data scratchpad sized equal to the bank’s row

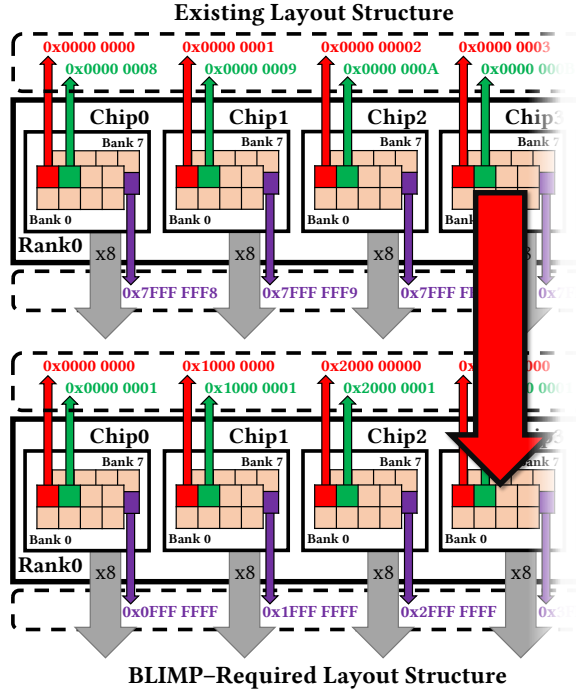


Figure 2: DRAM Hierarchy with baseline address mappings (top) vs suitable mappings for BLIMP (bottom).

buffer (1 KB). To load instructions into the buffer, read memory into registers, or write data from the scratchpad, the processor utilises a data transfer MUX that is directly tied to the bank's row buffer via a pseudo-buffer (aliased to the processor as register "v0").

To handle conflicts between accesses from the host cores and a bank core, a switch is placed before the bank address decoder that is controlled by a processor enable signal. This signal is switched by the host via the memory controller once computation at the bank is ready to begin. In the offloaded mode, the bank does not respond to out-of-bank memory requests; the bank core generates addresses that goes through the normal decode logic to access the required rows/columns and the corresponding bits of the local row buffer. After computation, control is handed back to the host after the signal is tied low. The overall architecture can be visualized via Figure 1.

However, bank cores can only access memory within their bank, and not others. As a result, it is the responsibility of the host to ensure that all data required in the offloaded computation is completely self-contained within each bank for the corresponding bank core to work independently. This placement of data is done explicitly in software by the host cores, incurring additional (offload) overheads, that is discussed in greater detail in the next section. Another consequence of this requirement is the different address mapping scheme that we need, as shown in Figure 2, requiring a data placement mechanism from the host that explicitly recognizes and lays out the data accordingly. For instance, as stated earlier for the normal hardware in Section 2.2, bytes at addresses x , $x + 1$, ..., $x + 7$ fall on banks of different chips, and the rank level parallelism

when storing/retrieving a 64 bit word fully utilizes the 64 bit wide channel. However, with the address mapping imposed by BLIMP, these addresses must fall on the same bank of the same chip, requiring that they be sent one-byte-after-another, since each is a x8 bit-wide chip. Consequently, the remaining 56 bits of the channel are idle (during relayout), while each of the 8 bits of a 64 bit word is transferred to a bank. This introduces additional overheads in data layout before offloaded computation on the bank core can start, and is also accounted for in the overheads in our evaluations. Other similar bank-level PIM engines (such as UPMEM[7]) also have similar restrictions and layout properties.

3.2 BLIMP-V: BLIMP with Vector Engine

While BLIMP supports a core per bank, providing as much parallelism in the offloaded computation as the number of banks (128 in our system), in several cases, this parallel code works with vectors/arrays. It is thus natural to explore the benefits of adding vector units to these bank-level RISC-V cores, to further optimize for the repeated operations across vector elements, apart from the bank-level parallelism. In Figure 1, our BLIMP-V platform allows for an additional row-buffer-sized vector computation engine for floating point and integer arithmetic. To support vector operations on the row buffer, two additional row-buffer sized registers are included, acting as vector registers (v1 and v2), as well as an appropriately sized crossbar (XBAR). These registers are fed data through the BLIMP transfer MUX and are operated on via a configurable amount of ALUs and FPU's. For this work and evaluation, we assume full width ALUs and FPU's on a 1KB row buffer (256 32bit ALUs and 256 32bit FPU's), and the vector instructions are sequenced through by the RISC-V core.

4 SOFTWARE FOR HETEROGENEOUS PLATFORMS

A key goal of this work is to automate the mapping of off-the-shelf application source code onto a heterogeneous platform within emerging and future servers that not only have traditional multi-core CPUs and vector engines, but also general purpose cores and vector engines at the memory banks, such as those described in the previous section. Today's compilers are quite adept at identifying regions within applications that are best suited for exploiting multicores or vector engines. However, they do not yet have the capabilities to determine what and when to redirect and offload to PIM computing engines. With PIM becoming a commercially viable hardware expansion in the near future [3, 20], compilers must adapt to this new computing environment. We explore how a traditional compiler framework (specifically in the context of LLVM directed at off-the-shelf applications programmed in C/C++) can be enhanced with a 4 step code-to-core flow process – identification, separation, compilation, and recombination – to work on heterogeneous PIM platforms.

4.1 Computation Domain and Region Identification

Current compilers support automatic code generation for scalar, vectorizable, and multithreaded application workloads. Identification of these program regions and transformation techniques to


```

1 void relayout_to_bank(
2     uint32_t bank_idx,
3     uint64_t region_size,
4     uint64_t* memory_region
5 ) {
6     uint64_t bank = bank_idx * 8;
7     for (uint64_t i = 0;
8         i < region_size / sizeof(uint64_t);
9         i += sizeof(uint64_t))
10    {
11        uint64_t data = memory_region[i];
12        for (uint64_t j = 0; j < 64; j+=8) {
13            memory_region[i + j] =
14            ((data & (0xFF00000000000000 >> j)) << j) >> bank;
15        }
16    }
17 }

```

Listing 1: Simple data relayout algorithm to offload an arbitrary region to a specific bank (as seen in Figure 2).

traditional hardware is well studied in compiler research [15, 36, 63] and is not a focus for this work. Instead, our goal is to incorporate these state-of-the-art techniques with a decidability factor to determine what, when, and where to execute program regions based on not just the potential speedups for those regions, but also the costs of performing and offloading such computations, which will impact program performance as a whole.

To achieve this, the compiler has to decide whether (i) parallelizable code should be run by the host cores or the RISC-V cores at the bank, and (ii) vectorizable code should be run on the host-side AVX engines or on the vector engines at the RISC-V cores close to the bank. This decision is based on the performance benefits vs. offload overhead trade-offs and we use a three step identification process for doing this as discussed below:

Step 1: Vectorizable? Vectorizable regions are typically those that involve iterations manipulating vector elements and we mainly focus on such program regions which obey the following invariants:

- Inter-loop data independence: No iteration determines the state of another element, value, or execution pattern.
- Non-shortable Iterations: When an iteration sequence begins, all iterations must assume executability.
- Maskable Conditions: If a branch is present in an iteration, it exists in all iterations and can be maskable.

If all these invariants hold, a region is deemed vectorizable and moves to Step 3, otherwise is forwarded to Step 2.

Step 2: Parallelizable? If an algorithm is not vectorizable, it has violated one of the vectorization invariants defined before. Recall however, that vectorization is simply a more strict check for parallelization. If a region is parallelizable, it can still be run on BLIMP cores or on the host cores. Using the same mechanism that Step 1 uses, we check to see if it will hold the following invariant:

- Inter-loop element independence: For a given iteration, no prior or future iteration determines the state/value/execution-pattern of the current iteration.

We are assuming parallelizable regions that do not need the use of locks, mutexes, or semaphores. If this invariant holds, the region is deemed parallelizable and proceeds to Step 3. Otherwise

the region is deemed not suitable for any offload, and is simply executed on the scalar core of the host CPU.

Step 3: Offloadable? Regardless of whether the region is vectorizable or parallelizable, it must meet an offloadability test: do regions satisfy strict BLIMP or BLIMP-V (BLIMP/-V) constraints and do offload overheads to/from BLIMP/-V outweigh potential gains? Recall BLIMP/-V cores operate on restricted and independent memory spaces (32MB in our configuration). Because of this, our framework must examine the following region attributes: (i) code size/span, (ii) input data size, (iii) working/scratchpad data size, and (iv) output data size. If the sum of all these attributes cannot be chunked into bank-sized “threads”, it cannot be offloaded. Of course, statically determining input, working, and output data sizes is a field of research on its own. When these sizes are categorically unknown, our framework utilizes conservative heuristic estimates (such as those used by auto-vectorizers) and/or manual pragmas (such as those used by auto-parallelizers). In the event that these estimates are not calculable due to insufficient information at compile time, we defer and embed this offload check to the at-runtime offload preamble (described further in Section 4.3). Given a region is deemed BLIMP/-V satisfiable at compile time, the next issue to tackle is that of offload cost and feasibility.

To relay the input (and subsequently the output) data, locally to (and from) the bank, one could employ (i) a sophisticated hardware mechanism/controller that re-stripes data as per BLIMP/-V requirements, or (ii) use the host core. As explained earlier, we assume no explicit hardware support for doing this, and so rely on the host cores. Such an algorithm for placing an arbitrary number of bytes into a bank within an arbitrary chip is given by Listing 1. The cost in time for relaying r bytes of data out to (o_B) and relaying data from (o_C) a single DDR bank on a 64-bit host can be expressed numerically as:

$$o_B(r) = \frac{r}{8} \times (t_r + 8t_w + 64t_{ops}) \quad (1)$$

$$o_C(r) = \frac{r}{8} \times (8t_r + t_w + 64t_{ops}) \quad (2)$$

where t_r is the time for a fetched memory read, t_w is the time for a committed memory write, and t_{ops} is the time to execute a logic (AND, ADD, SHIFT) operation.

These costs (data relayout to and from the BLIMP cores) should be lower than the potential speedups obtainable from the offload of the considered code region. This speedup is not only a function of the relative number of cores at the bank vs. the cores on the host, but also the speed of a single core – note that we assume only a relatively weak 200MHz RISC-V core at the bank vs. a Xeon Gold (2.2GHz) core at the host.

To estimate BLIMP (t_B) and CPU (t_C) execution times of the code segment under consideration, we need the total size of the working memory region in bytes, r , the number of operations or instructions per thread ops , and the parallelization factor or thread count, n :

$$t_B(r, ops, n) = o_B(r_i) + \frac{ops_m \times AMAT_B + ops_s}{n \times s_B} + o_C(r_o) \quad (3)$$

$$t_C(r, ops, n) = \frac{ops_m \times AMAT_C + ops_s}{n \times s_C \times c(n)} \quad (4)$$

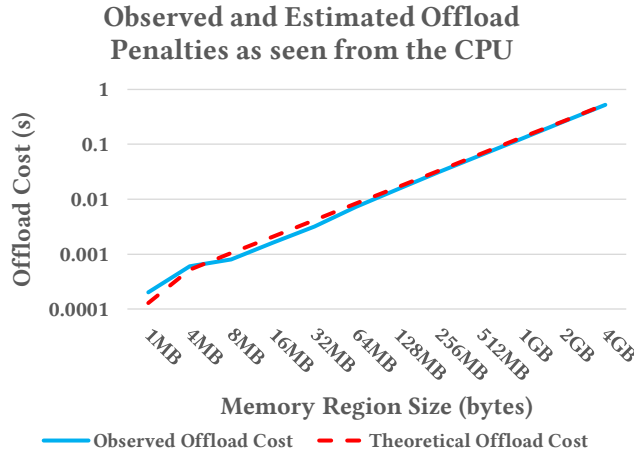


Figure 3: Theoretical vs real-world runtimes for varying region sizes, on log scale.

where r_i is the input data region size, r_o is the output data region size, ops_m is the number of memory operations, ops_s is the number of scalar operations, $AMAT_B$ and $AMAT_C$ is the average memory access time as seen by BLIMP and the CPU respectively, s_B and s_C is the speed of the BLIMP and CPU cores, and finally c is the contention factor (when accessing shared resources or memory). The contention for memory arises amongst the concurrent CPU cores trying to access memory, even if they are to different addresses. On the other hand, each RISC-V core in BLIMP/-V only accesses its individual bank, incurring no contention.

If t_B is estimated to be larger than t_C (with some configurable threshold), the compiler does not offload the code. Else, if the latter is larger, it offloads. The effectiveness of this offload decision by the compiler is clearly dependent on the accuracy of these attributes and costs calculated analytically as explained about. To illustrate the accuracy of our analytical models, in Figure 3, we compare the time taken for an offloaded region which works with a parameterized number of bytes on a combination of an actual Xeon Gold-based host processor and a cycle-accurate simulator of RISC-V bank cores – details of the experimental setup are described later in Section 5. From this figure, we can see that the compiler estimated cost for the offload (done statically without running the code) is quite accurate, within 12.6% of the actual costs at runtime, suggesting that this mechanism would work quite well in practice. The compiler uses this estimate and compares it with the potential benefits of the offload to decide whether to offload or not.

4.2 Implementing the Compiler Pass

As mentioned, the identification step described previously is easily integrated in a multi-pass compiler framework, such as LLVM, or by manual pragma identification. To fully support BLIMP/-V compilation and execution, a four step compiler pass is defined, as shown in Figure 4 and described below.

4.2.1 Step 1: Region Identification. The source code of the application is chunked per block and run through the check defined by

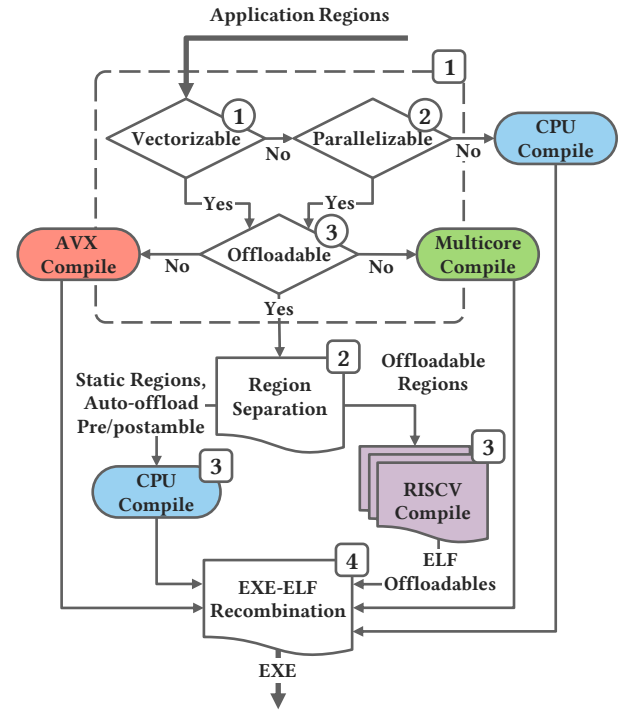


Figure 4: 3 step identification and 4 step compilation pass.

Section 4.1. Regions are tagged based on their offloadability and passed to the next step.

4.2.2 Step 2: Source Separation. If a block is deemed not offloadable, further compiler passes continue; otherwise, the block and its scope are transferred into another source file as a function stub. In place of this block, an auto-generated offload preamble (for at-runtime checks and input relayout), the offload syscall (with reference to the function stub, similar to a Remote Procedure Call invocation done by RPC compilers), and an auto-generated onload postamble (for output data recovery) is inserted.

4.2.3 Step 3: Code Compilation. Stubbed function blocks from the previous step are then run through a RISC-V compiler to compile the stub code to bank-level code. This is finalized to ELF format with all address references updated to correspond to the bank for which it executes in. Non-stubbed blocks are finalized and compiled using the host compiler.

4.2.4 Step 4: Application Recombination. The final RISC-V ELF(s) from the previous step are placed in the data segment of the now-compiled host application.

4.3 Runtime Operation

When the application begins to run on the host, bank ELF(s) from the data segment will be loaded into their respective compute banks. Normal application execution ensues until an offload preamble begins. Contingent on any runtime checks, the host will begin offloading any input data into as many banks as possible. If there

Table 1: Hardware and Simulation Parameters

System	Configuration
riscvOVPsim	RV64GCV, 0.9.x vector ISA, VLEN = 8192b 200MHz, Memory: 0x00000000:0x01ffff:rw
DRAMsim2	DDR4 2133 MHz; 64b datawidth; 4 channels, 2 ranks, 2 channels, 8 chips per rank, 8 banks per chip, tCK = 0.937ns, tRAS = 33.0ns, tRCD = 14.06ns, tCAS = 15ns, tWR = 15.0ns, tRP = 14.06ns [35]
Baseline System	Intel(R) Xeon(R) Gold 5220R CPU @ 2.20GHz L1: 32KB, L2: 1024KB, LLC: 36MB, RAM: 512GB
BLIMP DRAM	16GB total size; 128 BLIMP / BLIMP-V banks each 32MB in size, 8192b Row Buffer
BLIMP Bank	200Mhz BLIMP Core, 8192b Istr Buffer 8192b Data Scratchpad, 8192b R/W Buffer (v0)
BLIMP-V Bank	BLIMP Core, 2x8192b Vector Registers (v1, v2) Full Row Buffer-width ALUs and FPUs

are not enough banks, multiple rounds of chunked dispatches occur. Once each set of banks are loaded, a system call to the memory controller is invoked passing the offload function stub address as a means of indicating which bank/address to enable processing on. BLIMP/-V ELF execution will begin and run to completion. When complete, the onload postamble runs and retrieves any output data. This continues until all rounds of dispatch are complete.

At this point we make note of the following design choices; when the syscall is invoked to begin ELF execution, the core can block or continue execution when the banks are dispatched. In this work, we assume the host cores block.

5 EXPERIMENTAL SETUP

5.1 Design Space of Hardware Platforms

We evaluate and compare a hybrid BLIMP and BLIMP-V system against a current state-of-the-art baseline system with specifications listed in Table 1. To cover a variety of system capabilities, we consider the following six configurations:

- CPU or C: The baseline single core processor.
- CPU+AVX or CA: Configuration C with AVX512.
- MTCPU+AVX or MA: Configuration CA with 96 cores.
- CPU+BLIMP or CB: Configuration C with 128 BLIMP bank cores, totalling 4GB of operable space.
- CPU+BLIMP-V or CV: Configuration CB with added full row buffer width vector support per bank.
- CPU+AVX+BLIMP-V or CAV: Configuration CA with CV

There are five computation engines/regions in the above design space; the single core (CPU), the AVX engine (AVX), the multicore (MTCPU), BLIMP, and BLIMP-V. Each is best suited for executing an algorithmic/application behavior identified by our software system: offloadable vector regions are best suited for BLIMP-V execution, offloadable parallel regions are best suited for BLIMP execution, non-offloadable vector regions are best suited for the host AVX engine(s), non-offloadable parallel regions are best suited for a multicore, and finally static regions are best suited for single threaded CPU execution. Note that different portions of the same application can be suited for a different computation region, based on its parallelism, vectorizability, and offload costs.

Even though our software identifies which of the five engines to run a specific region of code, it may so happen that such an engine is unavailable in the considered hardware configuration. For example, if a non-offloadable vector region is identified on a CPU+BLIMP-V system, even through this region should ideally be run on a host AVX engine, it will be executed on BLIMP-V – helping us understand the impact of offload overheads. Table 2 shows where application regions are executed given a system configuration.

5.2 Evaluation Platform and Methodology

For the CPU, MTCPU and AVX engines, we run the relevant code region on real-world hardware as specified by Table 1. Since we do not have actual hardware for BLIMP and BLIMP-V, we have simulated those using riscvOVPsim[6] and DRAMSim2[56]. Using the riscvOVPsim simulator, we simulate runtimes based on an in-order, single stage, implementation of the processor. Since this simulator does not consider caches or memory delays, we opted to use DRAMSim2 to calculate the time required to access bank-level memory from the bank-level-core. Specifications of our DRAMSim2 and riscvOVPsim simulation parameters are also shown in the corresponding rows in Table 1.

To gather timing data for both the real-world and simulated architectures, benchmarks were instrumented with time fences before and after identified offload regions (just after the offload preamble and just before the reload postamble). Runtimes for CPU/MTCPU/AVX were then obtained by subtracting all fence times from the total real-world runtime. To gather timing data for BLIMP/-V, we compile a modified benchmark (from the perspective of a single bank) into RISC-V and run it through our simulation framework.

5.3 Workloads

Since our work is primarily targeting general purpose processors and their vector units, whether at the CPU or DRAM, we consider a number of datacenter-class workloads that have been previously studied for these engines¹. Our workloads include a number of applications from the SPEC CPU 2017 suite [14] and map-reduce paradigm based workloads in the Phoenix benchmark [2, 54].

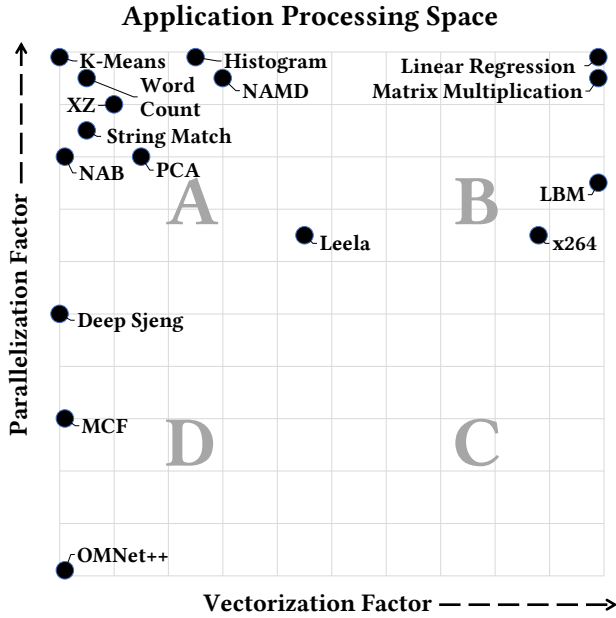
From the SPEC CPU 2017 suite, we pick nine applications (some were discarded since our software infrastructure for analysis and compilation required C/C++ source code). The evaluated SPEC benchmarks are MCF, OMNet++, x264, Deep Sjeng, Leela, XZ, NAMD, LBM, and NAB.

Additionally, we evaluate seven Map-Reduce applications from the Phoenix suite. These include Histogram (evaluated on 4GB of data), K-Means (100,000 points in 128 buckets), Linear Regression (4GB of data), Matrix Multiplication (of 2048x2048 matrices), PCA (5k x 5x points), String Match (500MB of data), and Word Count (2GB of data). The Map-Reduce framework creates multiple threads for parallelism in both the Map and Reduce phases, making these applications more suited to the MTCPU and/or BLIMP-V engines. However, the difference between these two kinds of engines is that the former can exploit parallelism in both the Map and Reduce phases, while the latter is limited in the amount of parallelism (mainly exploitable in Reduce phases) since each bank

¹We do not consider machine learning, graphics, and other workloads which are more suited to specialized compute engines (GPUs) or custom accelerators.

Table 2: Mapping of Application Regions to Hardware Given a System Specification.

Specification	Short	Non-offload Vector	Non-offload Parallel	Offload Parallel	Offload Vector	Static / Core
CPU+AVX+BLIMP-V	CAV	AVX	BLIMP	BLIMP	BLIMP-V	CPU
CPU+BLIMP-V	CV	BLIMP-V	BLIMP	BLIMP	BLIMP-V	CPU
CPU+BLIMP	CB	BLIMP	BLIMP	BLIMP	BLIMP	CPU
MTCPU+AVX	MA	AVX	MTCPU	MTCPU	AVX	CPU
CPU+AVX	CA	AVX	CPU	CPU	AVX	CPU
CPU	C	CPU	CPU	CPU	CPU	CPU

**Figure 5: The application processing space**

core is restricted to accessing the data within its own bank as has been discussed in a related recent work [53].

Figure 5 gives a pictorial representation of all 16 applications at a glance in terms of their exploitable parallelism and vectorization capabilities. When determining an application’s vectorization factor, we scale by the amount of time a parallel region spends in vectorizable computation. When determining an application’s parallelization factor, we consider the maximum number of threads that the application can support as well as its relative runtime. From this, we can pictorially visualize four quadrants of interest; applications that are highly-parallelizable and low-vectorizable (Region A), highly-parallelizable and highly-vectorizable (Region B), low-parallelizable and highly-vectorizable (Region C), and lastly low-parallelizable and low-vectorizable (Region D).

6 EVALUATION

We analyze application performance on all configurations listed under Table 1 for regions A-D identified in Section 4. We then show how different parameters affect the sensitivity of these results and the benefits of offloading to PIM.

We will be using the terms *perfect/under/over-satisfied* for the parallelization provided by BLIMP/-V in our discussions. With 128 banks (across 4GB) in our baseline configuration, we say an offload is perfectly satisfied if it can be offloaded to all 128 banks. If, however, there is no way to chunk or perform the computation across 128 cores, because of limited memory capacity, we denote the offloads’ parallelization as not satisfied, or under satisfied. In such cases, as will be explained, multiple rounds of the offload computation has to be performed. Lastly, if the application can run on 128 cores and has space left over (meaning each bank is not processing fully on 32MB of data), or not all BLIMP cores are utilized, we say that the offloads’ parallelization is over-satisfied. All averages shown are harmonic unless otherwise stated.

6.1 Region A - High Parallelism and Low Vectorization

From Figure 5 we see that 9 of 16 applications fall in this region including Histogram, K-Means, PCA, String Match, Word Count, Leela, XZ, NAMD, and NAB. For these applications, Figure 6 shows their runtime breakdowns when executed on 4 of the system configurations studied (from Table 2). We omit CPU and CPU+AVX runs as their relative runtimes are drastically different than the MTCPU+AVX (or MA) run. For this reason, we benchmark all systems against the MA execution. Despite their characteristics classifying them into Region A, the performance behavior is quite different across the configurations studied: (i) applications such as Histogram, PCA, String Match, and Word Count performing better on the host cores than on the BLIMP cores (illustrating that PIM is not always a panacea), (ii) applications such as K-Means, XZ, NAMD, and NAB performing better on the BLIMP cores than on the host cores (illustrating the benefits of PIM), and (iii) applications such as Leela performing equally well on the host and BLIMP cores, thus not requiring PIM. To better explain these results, we take an example application from each of these categories and drill down the results below.

6.1.1 Better on Host Cores. From Figure 6 we see 4 applications (Histogram, PCA, String Match, and Word Count) that perform 1.5× better on MA than on BLIMP/-V systems. Several factors – the relayout cost (described in Section 4.1), restrictive memory space (32MB local data per bank), the relatively slower clock speeds/capabilities (200MHz) of the bank-level processor – impact the benefits of offloading to PIM, and in these cases, one or more of these factors makes the host cores a better alternative for these applications compared to the PIM execution.

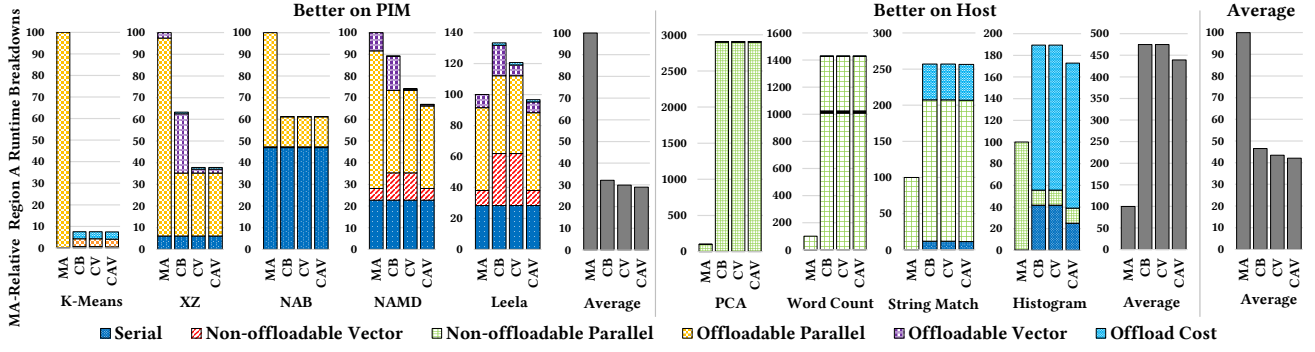


Figure 6: Breakdowns of all 9 Region A applications on 4 differing systems, with their runtimes normalized to that of the MA.

Take for example the Histogram application where a very simple loop performs 3 reads, 3 writes, and 12 ALU operations for each element across 4GB of data. Recall however that the offload algorithm defined in Section 4.1 runs similarly through each element and performs a number of reads, writes and operations before BLIMP computation can even begin. Because of this similarity, we see that in Figure 6-Histogram, 77% of the CAV runtime was spent offloading. When compared to the MA, the offload to BLIMP-V took 72.8% longer than the MA took to actually perform Histogram; pointing out that one may as well have done the computation on MA, despite CAV being 7.21× faster than MA for the computation itself.

In the other applications where the host cores were a better alternative, the offload cost was not the contributing factor. In fact, for the remaining applications in Region A, with exception to K-Means and String Match, the offload cost did not exceed 1.56% of any BLIMP/-V runtime. Instead, one of the serious limitations was the under-satisfied parallelism caused by the limited bank capacity (32 MB) and number of banks (128) explained earlier. For instance, take the Word Count application: since we do not know the distribution of words a priori, we can offload only about 15 MB of input data at a time (with roughly the same amount coming back as output) for each 32MB bank. Hence, in one round of offload, the 128 BLIMP cores can work on roughly 1.5 GB of input data. Since the input size is 2 GB in this application, it requires two offload rounds to be performed one after another (horizontal lines delineating the rounds are shown in the BLIMP/-V executions of Figure 6) – the first round perfectly-satisfied in parallelism and the second round over-satisfied in parallelism. With each round on BLIMP cores taking 973% and 257% of the MA execution, respectively, doubling the rounds results in exceeding the MA execution time by 13× for Word Count. This suggests that *one should not PIM* for Word Count, but this choice is determined by the problem size, bank size, and the number of banks on the given system (studied later in the sensitivity experiments).

The other factor, which materializes in some applications such as PCA, is the relative speed/capability of the RISC-V core with respect to the more powerful host cores. For instance, in PCA, the offloaded computation per core runs 34.8× slower than if the same computation were to run on a core of the host CPU. With 96-fold parallelism of the more powerful cores, the host overshadows

the 128 fold parallelism of the slower BLIMP cores, resulting in a slowdown of 95.41%.

6.1.2 Better on BLIMP Cores. We observe 3 applications from Region A (K-Means, XZ, and NAB) that perform 1.5× better on a BLIMP system than on the MA. In these applications, the BLIMP systems have typically satisfied or over satisfied parallelism, incur low offload costs, perform well due to the low memory latency at the bank level, or enjoy a combination of these.

For instance, XZ and NAB show maximum relayout penalties of 2.09% of the overall offloaded runtime, piling in comparison to the benefits. For K-Means specifically, 45.3% of it’s runtime was spent offloading. However when taken into consideration that K-Means performed 13.5× better than the MA, this overhead only accounts for 3.35% of the MA’s runtime.

When we consider the data being operated on with these applications, we find that all applications operate over-satisfied on the bank cores. For example, in K-Means with only 12.5% of the bank’s memory being utilized and operated on, each bank operates and calculates a single mean. With intermittent shuffling and low relayout communication, K-Means on BLIMP vastly outperforms the MA, despite the lower clock speeds of the former, even when the algorithm performs 140 internal iterations.

The remaining application (Leela) does considerably worse on a BLIMP system compared to the baseline. However, we still classify it as “Better on PIM” because if we had vector capability in the PIM cores, the resulting CAV run becomes more competitive, outperforming the baseline by 3.37%.

6.1.3 Comparing BLIMP and BLIMP-V. When we study Region A applications, we see that 6 of the 9 applications had low or no vectorization regions during their execution, thus resulting in no runtime difference between using a BLIMP or BLIMP-V architecture. For the three remaining applications (Leela, XZ, and NAMD), vectorization regions did not account for more than 8.4% of an application’s MA-relative runtime. In NAMD and Leela we see that what was originally best done on the host’s AVX engine is being taken on by BLIMP-Vs vector engines to benefit from the larger-width lower-latency accesses to the memory bank by the latter’s vector units. As a result, while Leela’s performance on BLIMP was worse than the baseline system by over 30%, by having a hybrid vector

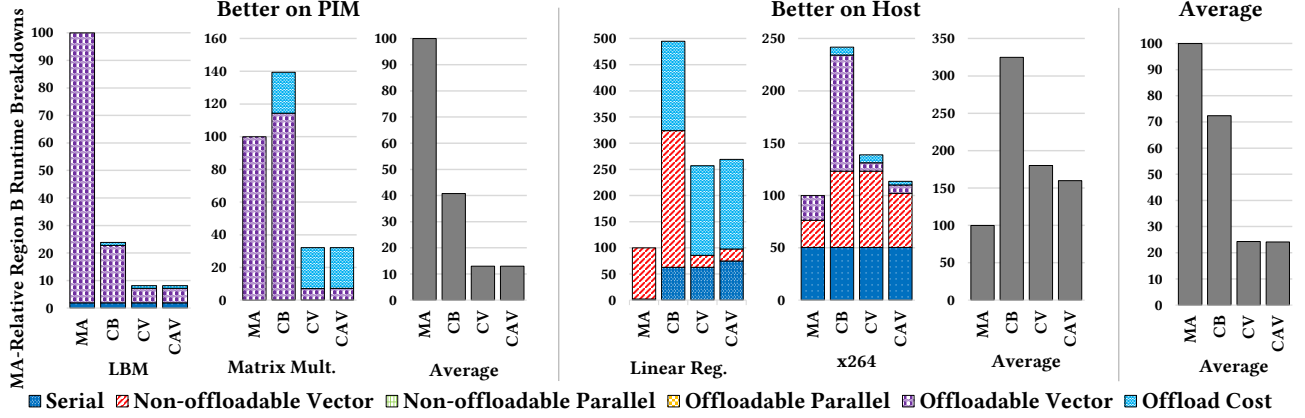


Figure 7: Breakdowns of all 4 Region B applications on 4 differing systems, with their runtimes normalized to that of the MA.

capability on both the host and PIM, the CAV configuration is able to beat the baseline.

6.2 Region B - High Parallelism and High Vectorization

There are 4 applications in Region B as shown in Figure 5; Linear Regression, Matrix Multiplication, LBM, and x264. We show their runtime breakdowns when executed on the different configurations in Figure 7 and discuss their suitability to PIM below.

6.2.1 Better on Host Cores. Linear Regression and x264 from Region B perform worse on the BLIMP/-V systems relative to the MA, with Linear Regression performing around 1.5× better on the MA than in PIM. Similar to Histogram from Region A, this application is a simple loop that, unlike Histogram, is vectorizable. Still, like its non-vectorizable counterpart, offloaded Linear Regression spends 63.6% of its runtime relaying out data, and when compared to the MA, simple relayout takes 41.5% longer than the MA spends executing the real program. Again, one may as well perform the computation on the host rather than offload this to PIM, despite the BLIMP-V cores accelerating in-region computation by 4.25× when compared to the MA.

While not as bad as Linear Regression, x264 does marginally worse (12.01%) than the MA when we delegate some of the computation to BLIMP-V. Overall, x264 spends roughly 85% of its runtime in a vectorizable or parallelizable region in single threaded mode, while on a multicore it spends 50% of its runtime in a vectorizable or parallelizable region.

However, this is not as simple as the region identification leads on. With most other applications, regions of high parallel or vector regions run near continuously, where the major CPU runtimes occur at the beginning and end of the parallel or vector compute region. This allows for nearly the entire application to be offloaded thus providing uninterrupted bank-level-compute. Unlike these other applications, the x264 application contains regions of parallelism and vectorization which are broken up by barrier-based CPU-bound compute for each P and I frames of the video, incurring multiple sub-rounds of offload compute.

6.2.2 Better on BLIMP-V Cores. Two applications (Matrix Multiplication and LBM) perform at least 1.5× better on BLIMP-V systems than the MA, despite the latter’s vectorization abilities. Let us consider Matrix Multiplication. Due to the wide width of the vector processor in BLIMP-V (16× larger than AVX512), full vector processing is significantly faster than the AVX engine of the host, despite being run on a clock which is roughly 10× slower. When we consider offloading Matrix Multiplication, we see it’s overhead is substantial, accounting for 77.7% of the offloaded runtime. Despite this, the major contributing factor to its speedup is the vector width processing capability of BLIMP-V and the low memory access time; we are able to load the entire row buffer of the bank into the vector register at the same time. However, to facilitate this, a special data layout is needed in the software before the offload. For each matrix row, it is placed sequentially in a row buffer and padded with zeros for the rest of the row buffer. Likewise, the transpose column of the matrix is placed sequentially in a similar manner after the rows. Although this data layout incurs higher memory capacity and also increases the offload time by a factor of two, Matrix Multiplication remains over-satisfied on BLIMP, with each bank only being 12.5% capacity utilized.

6.2.3 Comparing BLIMP and BLIMP-V. When we study Region B applications, we see that all applications had considerable differences between their BLIMP and BLIMP-V relative runtimes, which is to be expected considering each application spends at least 50% of it’s runtime in a vectorizable region. As a result, BLIMP vector-region runtimes exceeded MA vector-region runtimes by an average of 1.9×, with exception to LBM. When we consider the difference between BLIMP and BLIMP-V systems, we find that across all benchmarks, BLIMP-V outperformed BLIMP in vector-region runtime on average by 4.8×. When compared to Region A applications, it’s clear that for these applications, slow relative throughput induced by fetching data items one after another in BLIMP drastically penalizes the vectorization runtime.

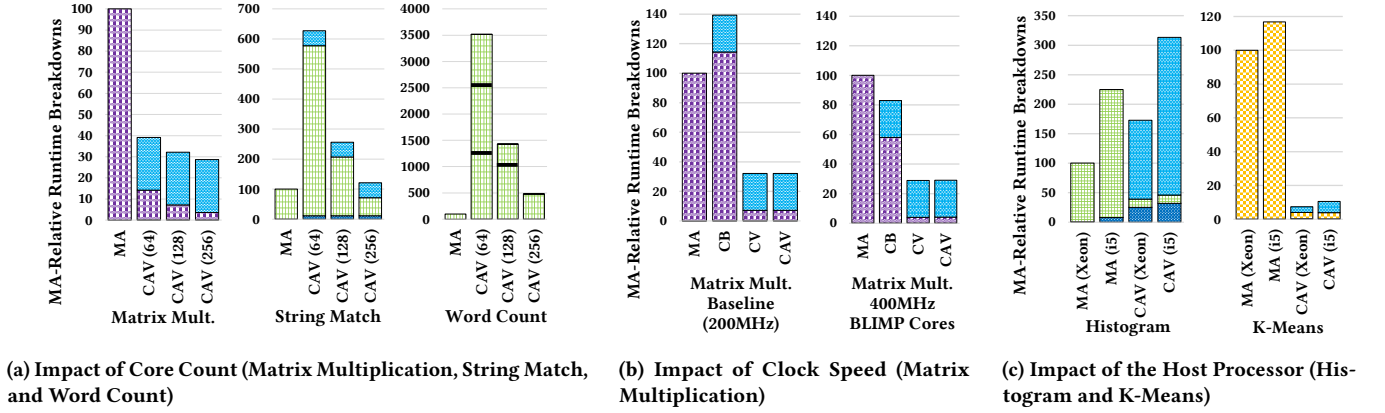


Figure 8: Sensitivity studies of the BLIMP core count, clock speed, and the host processor hardware.

6.3 Region C - Low Parallelism and High Vectorization

None of the applications under study fall into this region. In general, if an application experiences a significant portion of its runtime as being vectorizable, for example the Linear Regression benchmark, it is inherently also parallelizable, thus pushing the application into Region B.

6.4 Region D - Low Parallelism and Low Vectorization

When we examine the 3 remaining applications (MCF, OMNet++, and Deep Sjeng) from Region D, we find that these three applications are the top three applications that spend the most time in Serial or Static regions (99.9% OMNet++, 83.7% MCF, and 27.4% Deep Sjeng). It is no surprise then to know that for these applications, per-core performance and speed is the dominating performance characteristic. OMNet++ was in fact the only application studied that expressed zero vectorization or parallelization capabilities; as such, no proposed system accelerated its runtime and only raw single core CPU performance was exploitable.

For the remaining two applications, MCF and Deep Sjeng, the amount of data transferred ranged from a couple of megabytes to kilobytes, respectively, and between the two, relayout penalties only accounted for 0.2% and 0.7% respectively of their offloaded runtimes. Despite a low relayout penalty, to combat fast host cores, massively underutilized BLIMP/-V cores (that artificially over-satisfies the banks) was the only way to extract equal performance. Across all these benchmarks, neither the MTCPU or BLIMP/-V systems proved to provide speedups.

7 SENSITIVITY

Several factors in system parameters can impact the results observed in the previous section. We consequently study the impact of PIM bank count, PIM clock speeds, and altering the baseline processor to an average consumer CPU in today's market (instead of the high-end server-grade used earlier). For this study, we only evaluate the Phoenix benchmarks.

7.1 Impact of the Number of Banks

We change the number of PIM capable banks in our configurations to offer different degrees of parallelism. Figure 8a shows select benchmarks when varying PIM banks to 64 and 256, with the 128 bank study used in the previous section.

Across all benchmarks, we see that doubling the bank count accounts for only a 27.84% speedup overall but the offloaded region benefits from a 96.51% speedup. This is because of the static offload costs and non-offloadable, non-parallelizable, regions which do not scale with the increase in PIM cores.

Recall that for applications that are under-satisfied for BLIMP/-V compute, they required two offload rounds one after another due to a lack of bank-level cores to satisfy all chunked data regions. For Word Count specifically, not only was the parallelization factor fully satisfiable with 256 cores, but over-satisfied, leading each bank to receive less than 15MB of processing data each. Since in this new system only one round of offload occurred, and each bank performed computation with less data per-bank (compared to the baseline); simply doubling the bank-level core count allowed for a 198.7% faster execution. What this shows is that for applications that cannot be fully satisfied by the parallelism at the bank level, penalties are far greater than if their parallelism was fully or overly satisfied. Despite this massive increase in runtime however, Word Count is still better done on the host cores.

Halving the cores from 128 to 64 accounts for a 18.7% slowdown overall for the PIM configurations. As before, Word Count suffers significantly (59.4% slowdown), so much so that the offloaded implementation actually performed worse than the single-threaded version of the application.

7.2 Impact of the PIM Clock Speed

We investigate the effects of doubling the clock speed from 200MHz to 400MHz. Across all benchmarks, we find that doubling the bank clock frequency only accounts for a 28.35% speedup overall but the offloaded region benefits from a 73.20% speedup. One key application is Matrix Multiplication, shown in Figure 8b. When running the algorithm on BLIMP, we find that, overall, doubling the core frequency resulted in a 1.67 $\times$ speedup overall (becoming better

than doing everything on the host), though the offloaded region accounted for a $1.97\times$ speedup. Contrast this to when executed on BLIMP-V, an overall speedup of $1.11\times$ is achieved but only $1.81\times$ for the offloaded region. We attribute this disparity to the data needs of the wide vector engine in BLIMP-V vs the scalar engine of BLIMP. In the BLIMP execution, an order of magnitude less compute is being performed per memory access than does the vector engine in the BLIMP-V execution.

7.3 Common Consumer Host Cores

We also study the “to PIM or not” question for an Intel i5-3570K (4-core) desktop processor clocked at 3.40GHz, while retaining the same BLIMP/-V specifications. With a new host, PIM runtimes themselves do not change, rather, the time to offload and non-offloadable regions do. We find that for the i5, data relayout is roughly $1.9\times$ slower than that of the baseline. In general, when we compare the two hosts, we only see a 24.4% increase in runtime for offloadable applications.

Figure 8c shows 2 representative application breakdowns of the desktop-class (i5) and server-class (Xeon, baseline) runs; Histogram – where the host cores do better, and K-Means – where PIM does better. Recall for Histogram, the fundamental issue was that while doing offload, one may as well have performed the algorithm rather than offloading and letting the BLIMP/-V cores handle the execution; consequently, the desktop processor is not immune to this. However, for applications that were already solid BLIMP/-V contenders, the change in host impacts the offloaded performance minimally, as we see with K-Means. The offloaded difference between baseline and the i5 system is that of 30.4% but achieves a $9.4\times$ speedup against the common consumer-BLIMP system, as opposed to the $13.5\times$ speedup achieved by the baseline-BLIMP system. From this, we see that in applications where performing everything on the host was better, the percentage slowdown of the offloaded execution goes down on less powerful host systems, while applications which benefit greatly by offloading a significant portion to PIM are relatively less affected by host core changes.

8 RELATED WORK

8.1 Processing In Memory

While the idea of PIM has been around for a long time [21, 22, 25, 29, 38, 41, 45, 48, 49, 61], of late there has been a resurgence in interest. Broadly, work on PIM can be classified into two categories; (i) customized – where accelerators or logic optimized for specific applications or operations are incorporated into memory technology [11, 18, 19, 33, 34, 42, 55, 58, 60], and (ii) general purpose – where lightweight general purpose cores are embedded within the memory architecture [20, 51–53, 67]. We use an existing industry product [20] as the basis for one of the PIM architectures that is studied in this paper.

8.2 Application Offloadability

This research can be broadly classified into the following categories: (i) Offloadability based on qualitative benefits of PIM such as parallelizability, [51–53] (by leveraging the Map Reduce paradigm), and reduced memory latency [32], (ii) based on factors such as bandwidth savings [31], energy efficiency [40], thermal constraints

[47], and workload characteristics [12, 46], and lastly (iii) training a machine learning model to profile applications/application subsequently using it for predicting offloadability [17, 59, 65]. In contrast, our work combines both qualitative and quantitative metrics in the decision making process.

8.3 Compiler Support for PIM

Earlier specialized PIM solutions such as FlexRAM [38], and DIVA [21] provided compilers [24] [26]. However, most of the work today concentrates on HMC/HBM based implementations where the focus is on mitigating data movement bottlenecks [37, 50, 62]. Compiler based solutions for customized PIM architectures have been proposed for vector units embedded in the logic layer of HMC [8], for CGRA embedded in HMC with energy efficiency as a criterion [13], and for HMC based image processing accelerators [27]. Given that no such compiler exists for general purpose computing in DDR DRAM based solutions, our work aims to fill this gap.

8.4 Programming Models for PIM

CPU centric programming models are not a good fit for PIM. Efforts in this direction include a separate and customised programming model which takes into consideration the architecture, as well as scope for processing across the memory hierarchy [44] and HMC implementations [9, 32] with explicit offload regions. This may even include extending ISAs of existing architectures to support PIM [9]. There have been attempts at making everything transparent to the programmer by reworking existing parallel computing frameworks such as Apache Spark, MapReduce, [51–53] and MPI [10]. Alternately, compilers automatically identify and transform these regions appropriately to a form suitable for the targeted PIM architecture [28, 31, 64]. Our work falls into the last camp.

9 CONCLUDING REMARKS

This paper has taken on an important problem of where to perform computations in a growingly heterogeneous environment, where Processing-In-Memory (PIM) general purpose cores are alternatives to take on the load performed by the main CPU cores and their vector engines. We have developed a compilation framework on LLVM that takes different application characteristics – parallelizability, vectorizability, data sizes, and offload/relayout costs – in determining what, whether, when and how to offload computations to the PIM engines. Using a number of off-the-shelf C/C++ code, we have shown that PIM is not always a panacea, and a framework such as ours is essential in carefully selecting what needs to be performed where and how. The choice of hardware platforms – number of memory banks, relative speeds and capabilities of processor and PIM cores, can further influence the “to PIM or not” question.

ACKNOWLEDGMENTS

This work was supported in part by CRISP, one of six centers in JUMP, a Semiconductor Research Corporation (SRC) program sponsored by DARPA and NSF grants 1909004, 1714389, 1912495, 1629915, 1629129, 1763681.

REFERENCES

- [1] 2021. *AMD Bulldozer Processor Families*. Retrieved July 30, 2021 from <https://www.cpu-world.com/CPUs/Bulldozer/index.html>
- [2] 2021. *GitHub - kozyraki/phoenix: An API and runtime environment for data processing with MapReduce for shared-memory multi-core & multiprocessor systems*. Retrieved July 30, 2021 from <https://github.com/kozyraki/phoenix>
- [3] 2021. *HBM PIM | Technology | Samsung Semiconductor*. Retrieved July 30, 2021 from <https://www.samsung.com/semiconductor/solutions/technology/hbm-processing-in-memory/>
- [4] 2021. *Intel® Advanced Vector Extensions 512 (Intel® AVX-512) Overview*. Retrieved July 30, 2017 from <https://www.intel.in/content/www/in/en/architecture-and-technology/avx-512-overview.html>
- [5] 2021. *RISC-V International*. Retrieved September 27, 2021 from <https://riscv.org/>
- [6] 2021. *riscvOVPsim - Free Imperas RISC-V Instruction Set Simulator | Imperas - Embedded Software Development*. Retrieved July 30, 2021 from <https://www.imperas.com/riscvovpsim-free-imperas-risc-v-instruction-set-simulator>
- [7] 2021. *UPMEM | UPMEM is releasing a true Processing-in-Memory (PIM) acceleration solution*. Retrieved July 30, 2021 from <https://www.upmem.com/>
- [8] Hameeza Ahmed, Paulo C. Santos, João P. C. Lima, Rafael F. Moura, Marco A. Z. Alves, Antônio C. S. Beck, and Luigi Carro. 2019. A Compiler for Automatic Selection of Suitable Processing-in-Memory Instructions. In *2019 Design, Automation Test in Europe Conference Exhibition (DATE)*. 564–569. <https://doi.org/10.23919/DAT.2019.8714956>
- [9] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture. In *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*. 336–348. <https://doi.org/10.1145/2749469.2750385>
- [10] Mohammad Alian, Seung Won Min, Hadi Asgharimoghaddam, Ashutosh Dhar, Dong Kai Wang, Thomas Roewer, Adam McPadden, Oliver O'Halloran, Deming Chen, Jinjun Xiong, Daehoon Kim, Wen-mei Hwu, and Nam Sung Kim. 2018. Application-Transparent Near-Memory Processing Architecture with Memory Channel Network. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 802–814. <https://doi.org/10.1109/MICRO.2018.00070>
- [11] Marco Alves, Paulo Santos, Matthias Diener, and Luigi Carro. 2015. Opportunities and Challenges of Performing Vector Operations inside the DRAM. 22–28. <https://doi.org/10.1145/2818950.2818953>
- [12] Bahar Asgari, Ramyad Hadidi, Jia Shen Cao, Da Eun Shim, Sung-Kyu Lim, and Hyesoon Kim. 2021. FAFNIR: Accelerating Sparse Gathering by Using Efficient Near-Memory Intelligent Reduction. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 908–920. <https://doi.org/10.1109/HPCA51647.2021.00080>
- [13] Saambhavi Baskaran and Jack Sampson. 2020. Decentralized Offload-Based Execution on Memory-Centric Compute Cores. In *The International Symposium on Memory Systems (Washington, DC, USA) (MEMSYS 2020)*. Association for Computing Machinery, New York, NY, USA, 61–76. <https://doi.org/10.1145/3422575.3422778>
- [14] James Bucek, Klaus-Dieter Lange, and Jöakim v. Kistowski. 2018. SPEC CPU2017: Next-Generation Compute Benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering (Berlin, Germany) (ICPE '18)*. Association for Computing Machinery, New York, NY, USA, 41–42. <https://doi.org/10.1145/3185768.3185771>
- [15] Kuan-Hsu Chen, Bor-Yeh Shen, and Wu Yang. 2010. An automatic superword vectorization in LLVM. In *16th Workshop on Compiler Techniques for High-Performance and Embedded Computing*. 19–27.
- [16] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 27–39. <https://doi.org/10.1109/ISCA.2016.13>
- [17] Stefano Corda, Madhurya Kumaraswamy, Ahsan Javed Awan, Roel Jordans, Akash Kumar, and Henk Corporaal. 2021. NMPO: Near-Memory Computing Profiling and Offloading. *CoRR* abs/2106.15284 (2021). [arXiv:2106.15284](https://arxiv.org/abs/2106.15284) <https://doi.org/10.1109/DAC.2018.8465866>
- [18] Palash Das, Shivam Lakhotia, Prabodh Shetty, and Hemangee K. Kapoor. 2018. Towards Near Data Processing of Convolutional Neural Networks. In *2018 31st International Conference on VLSI Design and 2018 17th International Conference on Embedded Systems (VLSID)*. 380–385. <https://doi.org/10.1109/VLSID.2018.94>
- [19] Quan Deng, Lei Jiang, Youtao Zhang, Minxuan Zhang, and Jun Yang. 2018. DrAcc: a DRAM based Accelerator for Accurate CNN Inference. In *2018 55th ACM/ES-DAT/IEEE Design Automation Conference (DAC)*. 1–6. <https://doi.org/10.1109/DAC.2018.8465866>
- [20] Fabrice Devaux. 2019. The true Processing In Memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*, Cupertino, CA, USA, August 18–20, 2019. IEEE, 1–24. <https://doi.org/10.1109/HOTCHIPS.2019.8875680>
- [21] Jeff Draper, Jacqueline Chame, Mary Hall, Craig Steele, Tim Barrett, Jeff Lacoss, John Granacki, Jaewook Shin, Chun Chen, Chang Kang, Ihn Kim, and Gokhan Daglikoca. 2002. The Architecture of the DIVA Processing-In-Memory Chip. (09 2002). <https://doi.org/10.1145/514191.514197>
- [22] D.G. Elliott, M. Stumm, W.M. Snelgrove, C. Cocjaru, and R. Mckenzie. 1999. Computational RAM: implementing processors in memory. *IEEE Design Test of Computers* 16, 1 (1999), 32–41. <https://doi.org/10.1109/54.748803>
- [23] João Dinis Ferreira, Gabriel Falcao, Juan Gómez-Luna, Mohammed Alser, Lois Orosa, Mohammad Sadrosadati, Jeremie S Kim, Geraldo F Oliveira, Taha Shahroodi, Anant Nori, et al. 2021. pluto: In-dram lookup tables to enable massively parallel general-purpose computation. *arXiv preprint arXiv:2104.07699* (2021).
- [24] Basilio B. Fraguera, Jose Renau, Paul Feautrier, David A. Padua, and Josep Torrellas. 2003. Programming the FlexRAM parallel intelligent memory system. In *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2003, June 11–13, 2003, San Diego, CA, USA*, Rudolf Eigenmann and Martin C. Rinard (Eds.). ACM, 49–60. <https://doi.org/10.1145/781498.781505>
- [25] M. Gokhale, B. Holmes, and K. Iobst. 1995. Processing in memory: the Terasys massively parallel PIM array. *Computer* 28, 4 (1995), 23–31. <https://doi.org/10.1109/2.375174>
- [26] John Granacki, Mary Hall, Jeffrey Draper, Jeff Lacoss, and Jacqueline Chame. 2004. DIVA (Data Intensive Architecture). (06 2004), 404.
- [27] Peng Gu, Xinfeng Xie, Yufei Ding, Guoyang Chen, Weifeng Zhang, Dimin Niu, and Yuan Xie. 2020. iPIM: Programmable In-Memory Image Processing Accelerator Using Near-Bank Architecture. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 804–817. <https://doi.org/10.1109/ISCA45697.2020.00071>
- [28] Ramyad Hadidi, Lifeng Nai, Hyeojong Kim, and Hyesoon Kim. 2017. CAIRO: A Compiler-Assisted Technique for Enabling Instruction-Level Offloading of Processing-In-Memory. *ACM Transactions on Architecture and Code Optimization* 14 (12 2017), 1–25. <https://doi.org/10.1145/3155287>
- [29] M. Hall, P. Kogge, J. Koller, P. Diniz, J. Chame, J. Draper, J. LaCoss, J. Granacki, J. Brockman, A. Srivastava, W. Athas, V. Freeh, Jaewook Shin, and Joonseok Park. 1999. Mapping Irregular Applications to DIVA, a PIM-based Data-Intensive Architecture. In *SC '99: Proceedings of the 1999 ACM/IEEE Conference on Supercomputing*. 57–57. <https://doi.org/10.1109/SC.1999.10019>
- [30] Lei Han, Zhaoyan Shen, Duo Liu, Zili Shao, H. Howie Huang, and Tao Li. 2018. A Novel ReRAM-Based Processing-in-Memory Architecture for Graph Traversal. *ACM Trans. Storage* 14, 1 (2018), 9:1–9:26. <https://doi.org/10.1145/3177916>
- [31] Kevin Hsieh, Eiman Ebrahim, Gwangsun Kim, Niladri Chatterjee, Mike O'Connor, Nandita Vijaykumar, Onur Mutlu, and Stephen W. Keckler. 2016. Transparent Offloading and Mapping (TOM): Enabling Programmer-Transparent Near-Data Processing in GPU Systems. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 204–216. <https://doi.org/10.1109/ISCA.2016.27>
- [32] Kevin Hsieh, Samira Khan, Nandita Vijaykumar, Kevin K. Chang, Amirali Boroumand, Saugata Ghose, and Onur Mutlu. 2016. Accelerating pointer chasing in 3D-stacked memory: Challenges, mechanisms, evaluation. In *2016 IEEE 34th International Conference on Computer Design (ICCD)*. 25–32. <https://doi.org/10.1109/ICCD.2016.7753257>
- [33] Wenqin Huangfu, Xueqi Li, Shuangchen Li, Xing Hu, Peng Gu, and Yuan Xie. 2019. MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12–16, 2019*. ACM, 587–599. <https://doi.org/10.1145/3352460.3358329>
- [34] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. FloatPIM: In-Memory Acceleration of Deep Neural Network Training with High Precision. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. 802–815.
- [35] Micron Technology Inc. 2021. *Micron. DRAM Data Sheet*. Micron Technology Inc. Retrieved January 20, 2021 from <https://www.micron.com/products/dram/ddr4-sdram>
- [36] Alexey N Ivutin and Anna G Troshina. 2018. Use LLVM for optimization of parallel execution of program code on the certain configuration. In *2018 ELEKTRO. IEEE*, 1–6.
- [37] Mahmut Kandemir, Jihyun Ryoo, Xulong Tang, and Mustafa Karakoy. 2021. Compiler support for near data computing. 90–104. <https://doi.org/10.1145/3437801.3441600>
- [38] Yi Kang, Wei Huang, Seung-Moon Yoo, D. Keen, Zhenzhou Ge, V. Lam, P. Pattnaik, and J. Torrellas. 1999. FlexRAM: toward an advanced intelligent memory system. In *Proceedings 1999 IEEE International Conference on Computer Design: VLSI in Computers and Processors (Cat. No.99CB37040)*. 192–201. <https://doi.org/10.1109/ICCD.1999.808425>
- [39] Dimitris Kaseridis, Jeffrey Stuecheli, and Lizy Kurian John. 2011. Minimalist Open-Page: A DRAM Page-Mode Scheduling Policy for the Many-Core Era. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (Porto Alegre, Brazil) (MICRO-44)*. Association for Computing Machinery, New York, NY, USA, 24–35. <https://doi.org/10.1145/2155620.2155624>
- [40] Byoung-Hak Kim, Eui Cheol Lim, and Chae Eun Rhee. 2019. Exploration of a PIM Design Configuration for Energy-Efficient Task Offloading. In *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–4. <https://doi.org/10.1109/ISCAS.2019.8875680>

- 10.1109/ISCAS.2019.8702339
- [41] Peter M. Kogge. 1994. EXECUBE-A New Architecture for Scaleable MPPs. In *1994 International Conference on Parallel Processing Vol. 1*, Vol. 1. 77–84. <https://doi.org/10.1109/ICPP.1994.108>
 - [42] Mingu Kong, Min-Sun Keel, Naresh R. Shanbhag, Sean Eilert, and Ken Curewitz. 2014. An energy-efficient VLSI architecture for pattern recognition via deep embedding of computation in SRAM. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2014, Florence, Italy, May 4-9, 2014*. IEEE, 8326–8330. <https://doi.org/10.1109/ICASSP.2014.6855225>
 - [43] Boxun Li, Peng Gu, Yi Shan, Yu Wang, Yiran Chen, and Huazhong Yang. 2015. RRAM-Based Analog Approximate Computing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 34, 12 (2015), 1905–1917. <https://doi.org/10.1109/TCAD.2015.2445741>
 - [44] Elliot Lockerman, Axel Feldmann, Mohammad Bakhshalipour, Alexandru Stanescu, Daniel Sanchez, and Nathan Beckmann. 2020. Livia: Data-Centric Computing Throughout the Memory Hierarchy. 417–433. <https://doi.org/10.1145/33737376.3378497>
 - [45] K. Mai, T. Paaske, N. Jayasena, R. Ho, W.J. Dally, and M. Horowitz. 2000. Smart Memories: a modular reconfigurable architecture. In *Proceedings of 27th International Symposium on Computer Architecture (IEEE Cat. No.RS00201)*. 161–171. <https://doi.org/10.1109/ISCA.2000.854387>
 - [46] Lifeng Nai, Ramyad Hadidi, Jaewoong Sim, Hyojong Kim, Pranith Kumar, and Hyesoon Kim. 2017. GraphPIM: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 457–468. <https://doi.org/10.1109/HPCA.2017.54>
 - [47] Lifeng Nai, Ramyad Hadidi, He Xiao, Hyojong Kim, Jaewoong Sim, and Hyesoon Kim. 2018. CoolPIM: Thermal-Aware Source Throttling for Efficient PIM Instruction Offloading. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 680–689. <https://doi.org/10.1109/IPDPS.2018.00077>
 - [48] M. Oskin, F.T. Chong, and T. Sherwood. 1998. Active Pages: a computation model for intelligent memory. In *Proceedings. 25th Annual International Symposium on Computer Architecture (Cat. No.98CB36235)*. 192–203. <https://doi.org/10.1109/ISCA.1998.694774>
 - [49] D. Patterson, T. Anderson, N. Cardwell, R. Fromm, K. Keeton, C. Kozyrakis, R. Thomas, and K. Yelick. 1997. Intelligent RAM (IRAM): chips that remember and compute. In *1997 IEEE International Solids-State Circuits Conference. Digest of Technical Papers*. 224–225. <https://doi.org/10.1109/ISSCC.1997.585348>
 - [50] Ashutosh Pattnaik, Xulong Tang, Adwait Jog, Onur Kayiran, Asit K. Mishra, Mahmut T. Kandemir, Onur Mutlu, and Chita R. Das. 2016. Scheduling Techniques for GPU Architectures with Processing-In-Memory Capabilities. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation (Haifa, Israel) (PACT '16)*. ACM, New York, NY, USA, 31–44. <https://doi.org/10.1145/2967938.2967940>
 - [51] S. H. Pugsley, J. Jestes, R. Balasubramanian, V. Srinivasan, A. Buyuktosunoglu, A. Davis, and F. Li. 2014. Comparing Implementations of Near-Data Computing with In-Memory MapReduce Workloads. *IEEE Micro* 34, 4 (2014), 44–52.
 - [52] S. H. Pugsley, J. Jestes, H. Zhang, R. Balasubramanian, V. Srinivasan, A. Buyuktosunoglu, A. Davis, and F. Li. 2014. NDC: Analyzing the impact of 3D-stacked memory+logic devices on MapReduce workloads. 190–200.
 - [53] Siddhartha Balakrishna Rai, Anand Sivasubramaniam, Adithya Kumar, Prasanna Venkatesh Rengasamy, Vijaykrishnan Narayanan, Ameen Akel, and Sean Eilert. 2021. Design Space for Scaling-in General Purpose Computing within the DDR DRAM Hierarchy for Map-Reduce Workloads. In *Proceedings of the 18th ACM International Conference on Computing Frontiers (Virtual Event, Italy) (CF '21)*. Association for Computing Machinery, New York, NY, USA, 113–123. <https://doi.org/10.1145/3457388.3458661>
 - [54] Colby Ranger, Ramanan Raghuraman, Arun Penmetsa, Gary Bradski, and Christos Kozyrakis. 2007. Evaluating MapReduce for Multi-core and Multiprocessor Systems. In *2007 IEEE 13th International Symposium on High Performance Computer Architecture*. 13–24. <https://doi.org/10.1109/HPCA.2007.346181>
 - [55] Venkata Yaswanth Raparti and Sudeep Pasricha. 2018. DAPPER: Data Aware Approximate NoC for GPGPU Architectures. In *2018 Twelfth IEEE/ACM International Symposium on Networks-on-Chip (NOCS)*. 1–8. <https://doi.org/10.1109/NOCS.2018.8512159>
 - [56] Paul Rosenfeld, Elliott Cooper-Balis, and Bruce L. Jacob. 2011. DRAMSim2: A Cycle Accurate Memory System Simulator. *IEEE Comput. Archit. Lett.* 10, 1 (2011), 16–19. <https://doi.org/10.1109/L-CA.2011.4>
 - [57] Vivek Seshadri, Yoongu Kim, Chris Fallin, Donghyuk Lee, Rachata Ausavarungrun, Gennady Pekhimenko, Yixin Luo, Onur Mutlu, Phillip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry. 2013. RowClone: Fast and Energy-Efficient in-DRAM Bulk Data Copy and Initialization. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (Davis, California) (MICRO-46)*. Association for Computing Machinery, New York, NY, USA, 185–197. <https://doi.org/10.1145/2540708.2540725>
 - [58] Vivek Seshadri, Donghyuk Lee, Thomas Mullins, Hasan Hassan, Amirali Boroumand, Jeremie Kim, Michael A. Kozuch, Onur Mutlu, Phillip B. Gibbons, and Todd C. Mowry. 2016. Buddy-RAM: Improving the Performance and Efficiency of Bulk Bitwise Operations Using DRAM. *CoRR abs/1611.09988* (2016). [arXiv:1611.09988](https://arxiv.org/abs/1611.09988) <http://arxiv.org/abs/1611.09988>
 - [59] Gagandeep Singh, Juan Gómez-Luna, Giovanni Mariani, Geraldo F. Oliveira, Stefano Corda, Sander Stuijk, Onur Mutlu, and Henk Corporaal. 2019. NAPEL: Near-Memory Computing Application Performance Prediction via Ensemble Learning. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. 1–6.
 - [60] Srivatsa Rangachar Srinivasa, Wei-Hao Chen, Yung-Ning Tu, Meng-Fan Chang, Jack Sampson, and Vijaykrishnan Narayanan. 2019. Monolithic-3D Integration Augmented Design Techniques for Computing in SRAMs. In *IEEE International Symposium on Circuits and Systems, ISCAS 2019, Sapporo, Japan, May 26-29, 2019*. IEEE, 1–5. <https://doi.org/10.1109/ISCAS.2019.8702536>
 - [61] Harold S. Stone. 1970. A Logic-in-Memory Computer. *IEEE Trans. Comput.* C-19, 1 (1970), 73–78. <https://doi.org/10.1109/TC.1970.5008902>
 - [62] Xulong Tang, Orhan Kislal, Mahmut Kandemir, and Mustafa Karakoy. 2017. Data Movement Aware Computation Partitioning. In *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 730–744.
 - [63] Xinmin Tian, Hideki Saito, Ernesto Su, Jin Lin, Satish Guggilla, Diego Caballero, Matt Masten, Andrew Savonichev, Michael Rice, Elena Demikhovskiy, et al. 2017. LLVM compiler implementation for explicit parallelization and SIMD vectorization. In *Proceedings of the Fourth Workshop on the LLVM Compiler Infrastructure in HPC*. 1–11.
 - [64] Kanishkan Vadivel, Lorenzo Chelini, Ali BanaGoza, Gagandeep Singh, Stefano Corda, Roel Jordans, and Henk Corporaal. 2020. TDO-CIM: Transparent Detection and Offloading for Computation In-memory. In *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1602–1605. <https://doi.org/10.23919/DATE48585.2020.9116464>
 - [65] Dong Ping Zhang, Nuwan Jayasena, Alexander Lyashevsky, Joseph L. Greathouse, Lifan Xu, and Michael Ignatowski. 2014. TOP-PIM: throughput-oriented programmable processing in memory. In *The 23rd International Symposium on High-Performance Parallel and Distributed Computing, HPDC'14, Vancouver, BC, Canada - June 23 - 27, 2014*, Beth Plale, Matei Ripeanu, Franck Cappello, and Dongyan Xu (Eds.). ACM, 85–98. <https://doi.org/10.1145/2600212.2600213>
 - [66] Zhao Zhang, Zhichun Zhu, and Xiaodong Zhang. 2000. A Permutation-Based Page Interleaving Scheme to Reduce Row-Buffer Conflicts and Exploit Data Locality. In *Proceedings of the 33rd Annual ACM/IEEE International Symposium on Microarchitecture (Monterey, California, USA) (MICRO 33)*. Association for Computing Machinery, New York, NY, USA, 32–41. <https://doi.org/10.1145/360128.360134>
 - [67] Vasileios Zois, Divya Gupta, Vassilis J. Tsotras, Walid A. Najjar, and Jean-Francois Roy. 2018. Massively Parallel Skyline Computation for Processing-in-Memory Architectures. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques (Limassol, Cyprus) (PACT '18)*. Association for Computing Machinery, New York, NY, USA, Article 1, 12 pages. <https://doi.org/10.1145/3243176.3243187>